ConnectTheDot

A Simple & Interactive Dot Connection Activity for Children

Name	Bhumika Tiwari
Email	bhumitiwari17@gmail.com
GitHub	https://github.com/Bhumika-17
Prototype Code Repo	https://github.com/Bhumika-17/Sugarizer-GSOC-2026-Prototype
Pawn Activity Link	https://youtu.be/Md8-rltxvCY
Demo Video Link	https://youtu.be/FHZRLaf0F24
University	Faculty of Engineering & Technology, GLS University, Ahmedabad, Gujarat, India
Program	B.Tech Computer Science & Engineering (specialization in Cybersecurity)
Language	English (first working language), Hindi (mother tongue)
Time zone	Indian Standard Time (UTC+5:30)
Project Size	Medium
Organization	Sugarizer, Sugar Labs

About Me

I am a final year Computer Science student who enjoys exploring systems from both a technical and user perspective. I like understanding how things work internally, but I care just as much about how they are experienced by the end user. I have built projects like a cyber triage tool that combines disk analysis with machine learning, as well as applications focused on usability and design. I am constantly learning, experimenting, and refining my skills, and I am excited by opportunities where I can contribute meaningfully while growing as a developer.

I focused on understanding the Sugarizer codebase in depth, exploring how the Presence API, Datastore API, Journal lifecycle, and RequireJS fit together to support collaborative activities. As a result, I built a working prototype of ConnectTheDot, which was my first hands-on contribution.

This process helped me move beyond surface-level familiarity and actually work within the system, giving me confidence in navigating the codebase and contributing meaningfully going forward.

Past Contribution PR: https://github.com/llaske/sugarizer/pull/2037

I found and fixed a bug in the Paint activity where the eraser wasn't drawing properly for collaborators in shared sessions. The issue was that onMouseDrag was sending the current cursor position instead of the previous one, so others would see broken dots instead of a smooth stroke.

Relevant Skills

- JavaScript and modular frontend development (VanillaJS) with CSS and HTML
- Understanding of Sugarizer architecture: Presence API, Datastore API, Journal lifecycle
- Asynchronous programming and event-driven logic
- Strong debugging and codebase navigation skills
- UI/UX-focused application development
- Git/GitHub for collaborative development
- Problem-solving with end-to-end implementation (ConnectTheDot prototype)

Time Commitment and Availability

I will be available full-time during the GSoC period. My university semester concludes before the coding period begins. I have no major commitments during this window. I will be working from India, where I reside, for the entire duration of the program.

Community Contact

During the two weeks I spent building the prototype, I joined the sugar-devel mailing list and read through recent threads to understand how the community discusses contributions and reviews PRs. I have shared the ConnectTheDot prototype with the mentor in the discord channel and am ready to discuss scope, architecture, and priority during the Community Bonding period.

How I Handle Feedback

I study Cybersecurity, where peer review of findings is standard practice. You learn quickly that the goal is a correct result, not credit for your first attempt. I approach code review the same way. If a mentor tells me my architecture is wrong, my first question is what the right one looks like, not a defence of my original choice. I have built the three mode modules as fully encapsulated units precisely so that any architectural change is isolated and does not require rewriting the Sugar shell.

Chosen Project Idea

ConnectTheDot is a Sugar Web Activity to be built around a simple idea: **children interact by connecting dots on a shared grid.**

3.1 Draw Mode: Free Canvas

- A simple dot grid becomes a space to draw, experiment, and build shapes freely
- Journal Backup to preserve the game states.
- Real time collaboration through the Presence API lets multiple children draw together on the same canvas

3.2 Connect Mode: Discover the Shape

- Numbered dot puzzles where children reveal shapes step by step
- Shapes scale smoothly using relative positioning across different screen sizes
- Designed to expand with more puzzles over time

3.3 Game Mode: Capture the Space

- A territory-based game played on the same dot grid
- Players move across the canvas to capture space and build control
- Can be played against an AI or another player in shared sessions
- Gentle mechanics where collisions stop movement instead of penalizing
- A short-timed game that encourages strategy, exploration, and fun.

3.4 Proposed Enhancements *(optional & subject to mentor feedback and approval)*

If the core three modes are completed and stabilized, and if time permits, I would like to explore the possibility of adding the following, subject to mentor feedback and approval:

Symmetry Mode

- A mirrored drawing experience where actions on one side of the grid are reflected on the other.
- Helps children explore spatial relationships and pattern recognition.
- Uses the same dot-grid interaction and line-connection mechanics as Draw Mode.
- Maintains consistency with the existing fill behaviour and overall system design
- Will only be implemented if it aligns with the project direction and mentor guidance
- My inspiration is from: https://toytheater.com/mirror/

The Prototype [code] [demo video] [tutorial]

All modes share a unified dot-grid interaction and fill mechanism derived from Draw Mode. UI is intentionally minimal to avoid distracting children. It supports both pointer-based interaction and keyboard-controlled movement for accessibility across devices

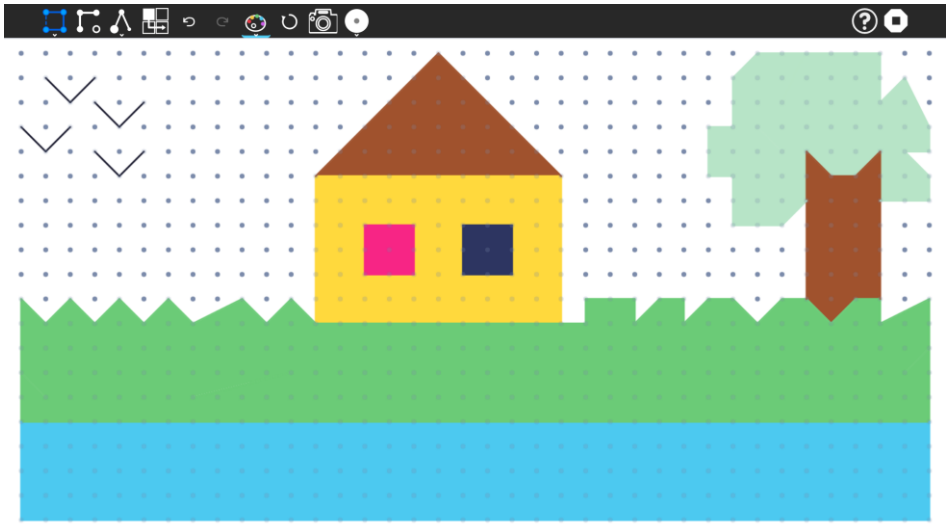


Figure 1 : Draw Mode Prototype

1. **Draw Mode**
   - Scalable dot grid; closed loops auto-fill with color
   - Smooth drawing with real-time preview while connecting dots
   - Accurate region filling for complex shapes
   - Undo/redo support; real-time collaboration via Presence API
   - Efficient grid handling for fast and consistent performance

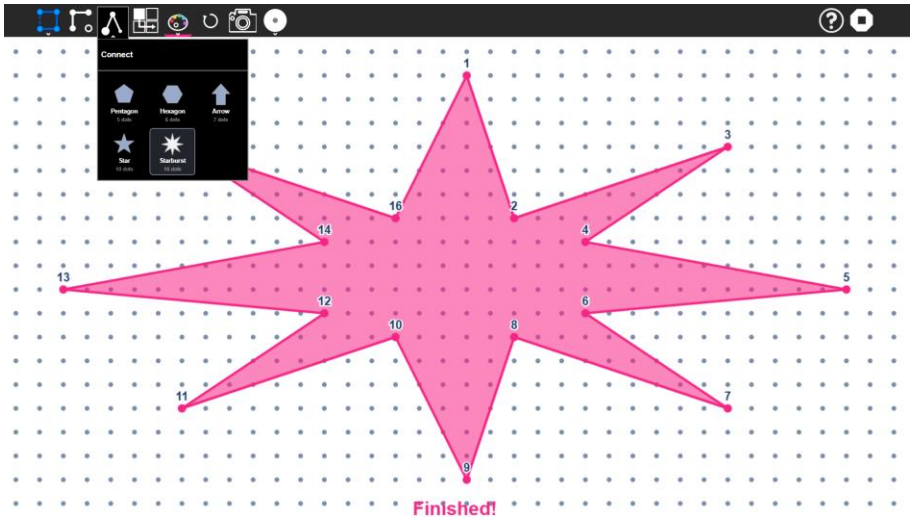


Figure 2 : Connect Mode Prototype

2. **Connect Mode**
   - Sequential dot puzzles revealing shapes step by step
   - Guided interaction (only next valid dot active)
   - Resolution-independent rendering using fractional coordinates
   - Multiple shapes with completion feedback
   - Easily extendable with new puzzles

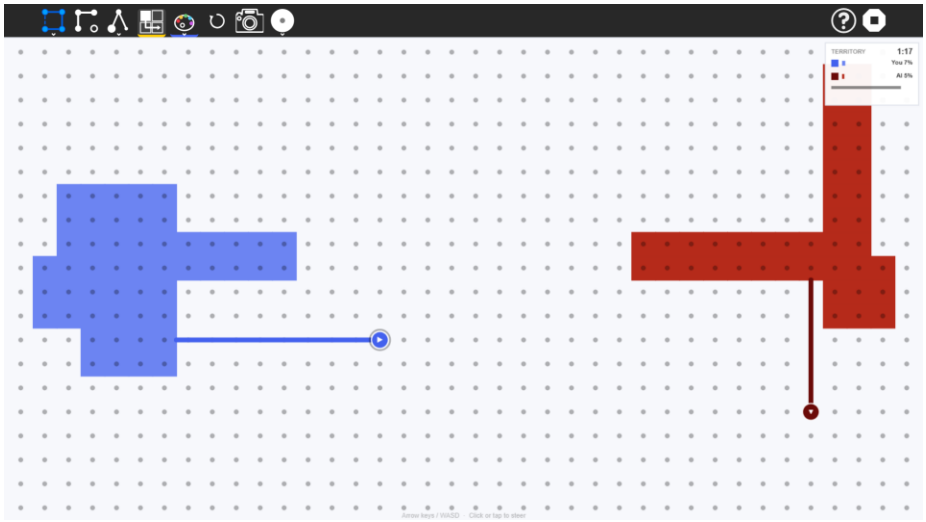


Figure 3 : Game Mode Prototype

3. **Game Mode**
   - Territory-based gameplay on the same dot grid
   - Capture areas by forming closed loops
   - Correct handling of enclosed regions
   - Simple AI opponent for single-player mode
   - Consistent performance with fixed timestep loop
   - Real-time multiplayer via Presence API

Prototype is implemented with the three modes, colour picker, reset board, save to journal and multi-player functionalities.

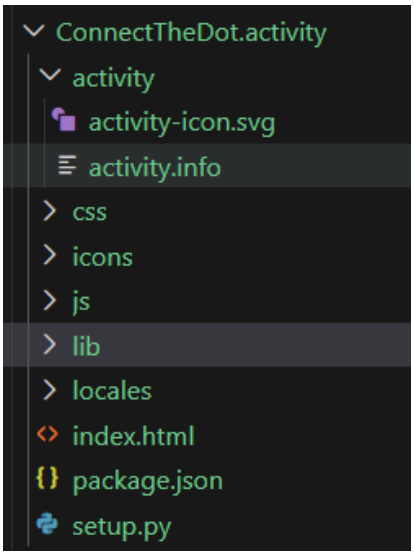
The core codebase is largely complete: Draw, Connect, and Game modes are fully implemented with working BFS logic, AI, real-time sync, and Sugar integration. Core modules handle lifecycle, rendering, interaction, localization, and collaboration.
Remaining work focuses on integration and polish: server registration, expanding Connect Mode content, adding tutorials, validating end-to-end multiplayer, and final performance optimization with deployment readiness.

Technical Approach

ConnectTheDot follows the standard Sugar Web Activity structure. The main file (activity.js) coordinates mode switching, handles the Sugar API lifecycle, and routes Presence messages.

The logic is split into three independent modules: pokpok-grid.js (Draw Mode), number-grid.js (Connect Mode), and game-mode.js (Game Mode). Each module exposes a simple API (init, destroy, setColor, clearBoard, getState, loadState) and works independently, making the code easy to maintain and extend.

The activity is built using **Vanilla JavaScript** with ES5-compatible AMD modules, following Sugar Web conventions, and uses the **HTML5** Canvas API, requestAnimationFrame, pointer/touch events, and the sugar-web library with **CSS** for beautification.



File Structure of Activity

API Calls - Exactly What I Used

I want to be specific about this, because it tells you whether I have actually read the API. These are the exact calls in my prototype code:

- *activity.setup()* : binds stop/pause lifecycle, colorises the toolbar icon with the user's XO colours
- *env.getEnvironment(callback)* : reads the user's language for l10n.init() and the objectId to check for an existing Journal entry
- *DatastoreObject.loadAsText(callback)* : reads saved JSON state on reopen
- *DatastoreObject.setDataAsText(json)* + *.save(callback)* : persists state on stop
- *datastore.create(metadata, callback, data)* : exports the canvas as a PNG to the Journal
- *presencepalette.PresencePalette* : renders the network sharing button UI
- *activity.getPresenceObject(callback)* : gets the presence network object
- *network.createSharedActivity(id, callback)* : creates a new shared session when the host shares
- *network.onDataReceived(callback)* : handles incoming messages from other users
- *network.sendMessage(sharedId, message)* : broadcasts line, colour, or game state delta to peers
- *l10n.init(language)* + *l10n.get(key)* : loads the correct language file and retrieves localised strings

All three modes implement a consistent getState() / loadState() mechanism for Journal persistence. The activity state is stored as a single JSON object, capturing the current mode and relevant drawing or game data, ensuring seamless save and restore across sessions.

Deliverables

Midterm

- Sugarizer activity running reliably on the server
- Connect Mode expanded with more shapes and responsiveness
- Guided tutorials implemented across all modes for onboarding clarity
- Real-time multiplayer (Presence) working with consistent state sync
- End-to-end functionality tested and ready for midterm evaluation

→ Result: a complete, usable, and collaborative activity with all core systems in place

Final Deliverables

- Verified cross-platform support
- Full localization coverage with clean l10n integration (en, es, fr, etc.)
- Edge cases resolved across gameplay, resizing, and mode transitions
- Clean, maintainable codebase with documentation and contribution guide
- Final PR submitted, iterated on, and ready for merge

→ Result: a polished, production-ready activity that is robust, scalable, and easy to extend

Timeline

➔ Throughout the timeline, I will maintain regular communication with the mentor and actively incorporate their feedback, as it is a top priority.

Phase 1: Core Integration & Features (Weeks 1–4)

- Week 1: Run the activity inside the Sugarizer server and fix integration issues
- Week 2: Expand Connect Mode (more shapes, responsive scaling)
- Week 3: Implement Tutorials
- Week 4: End-to-end Presence testing with two users, fix sync issues

Midterm Evaluation

- All planned deliverables completed and ready for review

Phase 2: Optimization & Validation (Weeks 5–8)

- Week 5: Performance tuning
- Week 6: Testing Completely
- Week 7: Complete localization audit
- Week 8: Test edge cases across all modes and transitions

Buffer & Stabilization (Weeks 9–10)

- Fix bugs from real usage
- Re-run performance checks

Final Phase (Weeks 11–12)

- Week 11: Clean code, write README, prepare PR
- Week 12: Submit PR, respond quickly to reviews, final validation

Risks and Mitigations

Risk	Mitigation
Presence state desync when a new user joins mid-session: When a new user joins an active session, incremental updates may arrive before the full state, causing an inconsistent canvas.	On the shared event, the host immediately sends a complete getState() snapshot. Joining clients ignore incremental updates until the snapshot is received.
Journal restore fails silently across Sugarizer version: Differences in datastore behavior (e.g., callback timing or metadata) may cause loadState to receive invalid data and fail without visible errors.	Validate loaded data before use and add a fallback to initialize a clean state if restore fails. Test save/restore against the latest Sugarizer build.
BFS flood fill blocks the main thread on large canvases: Full-canvas getImageData() and repeated BFS recomputation can freeze the UI, especially during resize.	Cache the mask canvas and update it only on new strokes. Split fill recomputation across frames using requestAnimationFrame to avoid blocking.

How will it impact Sugar Labs?

ConnectTheDot adds a playful, multi-dimensional activity to Sugarizer that blends creativity, logic, and light strategy in one place. Instead of a single-purpose tool, it offers drawing, puzzle-solving, and gameplay, keeping children engaged while learning in different ways.

Built with collaboration at its core, it turns individual interaction into shared experiences, reinforcing Sugar Labs' vision of social learning. The activity is lightweight and optimized for low-resource environments, staying true to Sugar's accessibility goals. Its modular design also makes it easy for future contributors to extend, whether by adding new puzzles, improving gameplay, or introducing new modes.

In short, it is not just a new activity, it is a flexible, extensible learning space that can grow with the community.

Post-GSoC Plans

I'm excited to continue contributing to Sugarizer well beyond GSoC. The project's mission of creating accessible learning tools for children, especially in constrained environments, truly resonates with me and I'm eager to keep building something meaningful here.

I plan to actively enhance ConnectTheDot with new features, improvements, and refinements, while also expanding my contributions across other parts of the codebase as I deepen my understanding of the system. Over time, I'm looking forward to taking on greater responsibility, collaborating more closely with the community, and playing a more impactful role in driving the project forward.